

Prise en main du robot ROSKit

Documentation Technique

13 mai 2026

Table des matières

1	Interagir avec le robot	2
1.1	Composition du robot	2
1.2	Comment allumer le robot	2
1.3	Télécommande	3
1.4	Connexion Wifi et SSH	4
2	Test de Navigation Autonome Simple	8
2.1	Objectif	8
2.2	Création du package ROS2	8
2.3	Architecture du programme	9
2.3.1	Publisher	9
2.3.2	Subscribers et callbacks	9
2.3.3	Timer	9
2.4	Odométrie — position et orientation	9
2.5	Traitement du LiDAR	10
2.6	Logique de navigation	10
2.7	Code source	10
2.8	Vérifications	11

1 Interagir avec le robot

1.1 Composition du robot

Le robot est équipé d'un :

- Scout Mini / Bunker Mini
- Asus NUC 15 Pro CRK
- Router Teltonika RUT956
- GPS : simpleRTK2B Ardu Simple
- IMU : Phidgetspatial
- RealSense D435
- LiDAR : Robosense Helios 16P

1.2 Comment allumer le robot

Appuyez sur le bouton de démarrage à l'arrière du robot. Le bouton s'allumera en rouge et l'écran affichera la batterie et le voltage.



FIGURE 1 – Arrière du Robot

1.3 Télécommande

Pour télécommander le robot, prenez la manette. Vous disposez de deux joysticks et 4 interrupteurs. Les interrupteurs n°2 et 3 ont trois crans, les autres en ont que 2. En plaçant tous les **interrupteurs en position haute**, maintenir les deux boutons power pendant quelques secondes. Des sons devraient retentir, la télécommande est allumée.



FIGURE 2 – Télécommande

Déplacement : Abaissez d'un cran le interrupteur n°2 et n°3. Le n°2 change le mode de communication du scout mini. En position haute, le scout mini communique en BUS sur le CAN0 avec la partie haute du robot, l'ordinateur de bord. Quand l'interrupteur est abaissé d'un cran, le robot ne reçoit d'ordre que de la télécommande. Le n°3 gère la vitesse. Additionnelement, le n°4 permet éteindre ou d'allumer les feux avant.

Vitesse (3ème interrupteur) :

- Position milieu : Vitesse lente
- Position haute : Vitesse medium
- Position basse : Vitesse haute

1.4 Connexion Wifi et SSH

Pour travailler sur le robot, il vous faut attendre quelques secondes que le router s'allume. Chercher le réseau du robot en wifi. Le nom du robot est inscrit sur le robot, il devrait s'appeler **roskit_ "numéro du robot"**.

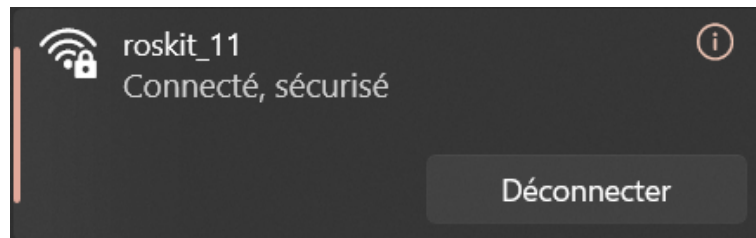


FIGURE 3 – Connexion Wifi au robot

Si ce nom n'apparaît pas c'est que **le robot a été réinitialisé**, il faut aller sur le manuel d'utilisation et chercher le nom correspondant au robot. Une fois connecté au robot, nous allons nous connecter à internet.

Connection internet : Pour se connecter à internet, on va se rendre sur <https://192.168.1.1>. Les identifiants sont dans le **manuel** section 6.1 : Route Page.

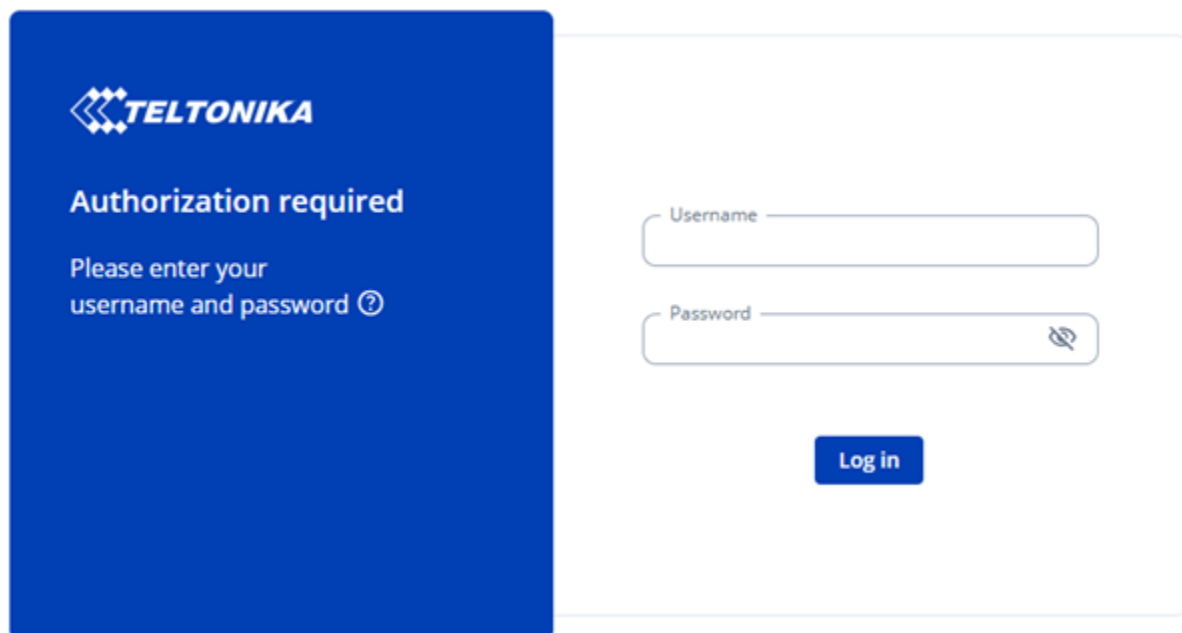


FIGURE 4 – Page d'accueil de teltonika

Une fois sur le site, allez dans Network, Wireless puis SSIDs. Ici :

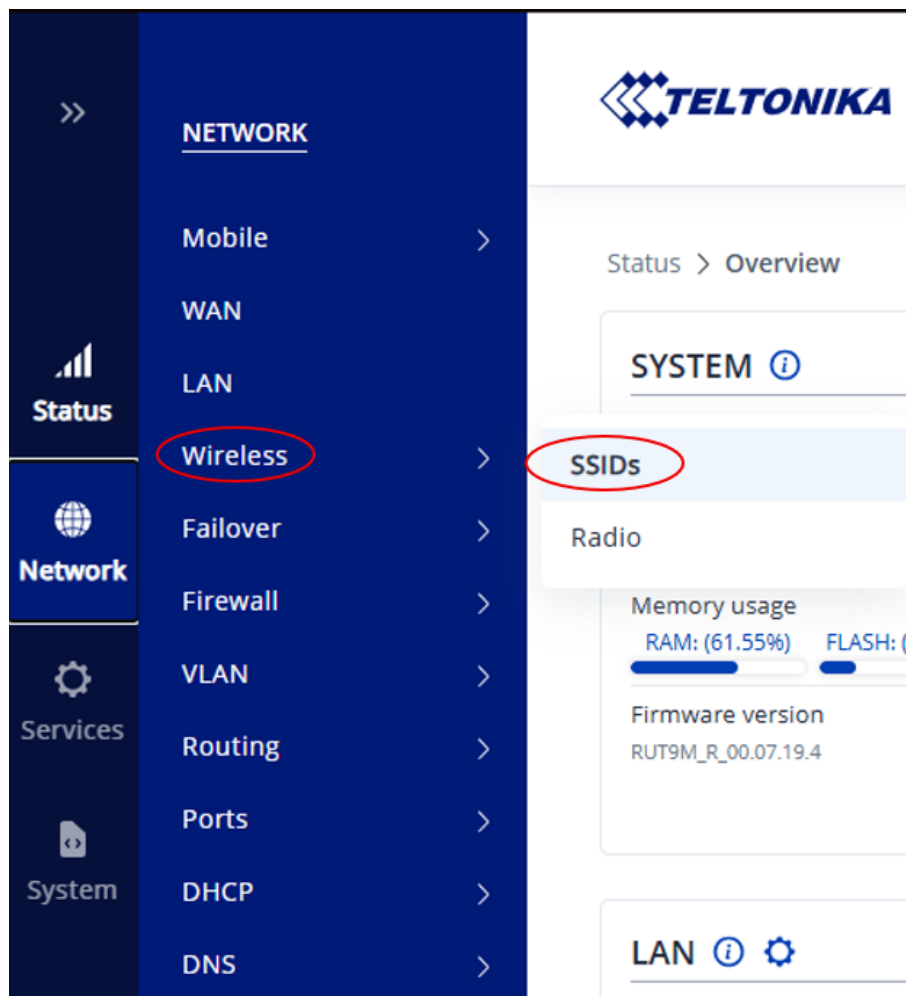


FIGURE 5 – Chemin d'accès SSID

Puis cliquez sur **Scan 2.4GHz**, sélectionnez la connexion que vous voulez utiliser (Unice Hotspot). Chaque robot ne peut-être connecté qu'à **une seule autre connexion**.

Pour utiliser Unice Hotspot : connectez vous dessus. Puis allé ici :

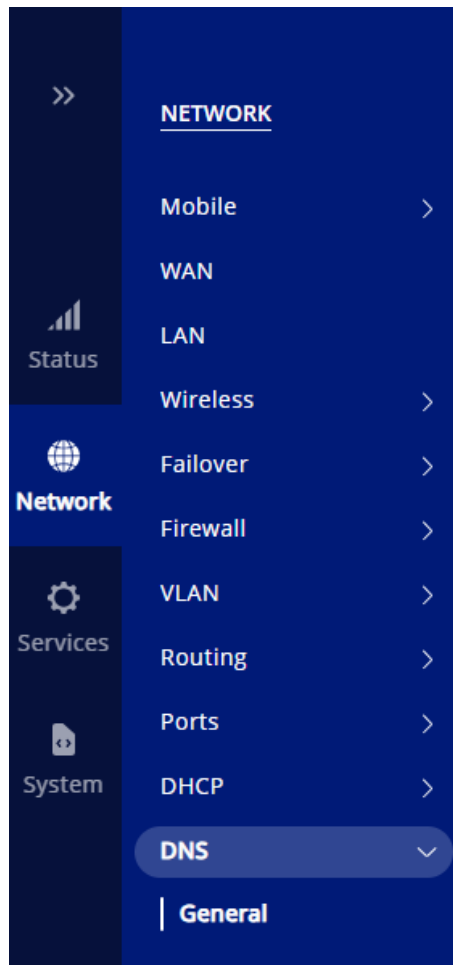


FIGURE 6 – Option DNS

Cherchez l'option Rebind Protection, décochez la. Une fois connectez, essayez de vous connecter à n'importe quel site internet, on vous redirigera vers la page Unice usuelle.

Connexion SSH :

1. Via noVNC : <https://192.168.1.102/>, le mot de passe est ROSKit pour tout les robots.. Vous êtes connecté sur l'ordinateur de bord du robot. Sur ce site vous aurez tous les **retours visuels des manipulations** de ce TP. Ouvrez un terminal (*voir figure 6*), vous pouvez taper « code . ». Une fenêtre VSCode devrait s'ouvrir.

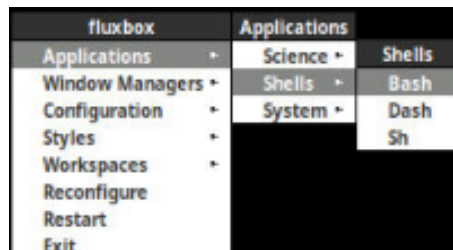
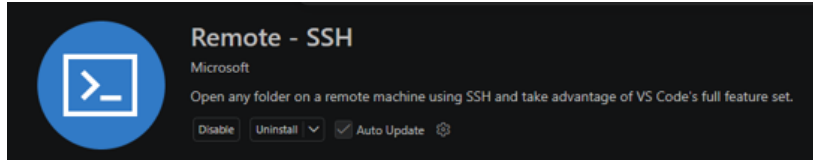
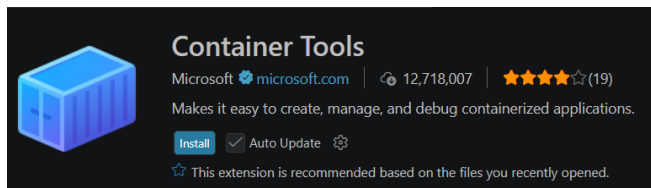


FIGURE 7 – Ouvrir un shell

2. Via VSCode Remote-SSH : Téléchargez Remote - SSH et Container Tool (voir figure 7) sur VSCode



(a) Extension Remote - SSH



(b) Extension Container Tool



(c) Option VSCode

FIGURE 8 – Extensions Information

Dans les options de VSCode on retrouve **3 sections** intéressantes (voir figure 7.c). La première pour se déplacer dans nos **fichiers** (en haut), la deuxième est le **remote explorer** (au milieu) et la troisième l'**accès au docker** (en bas). Pour l'instant, rendez-vous sur le remote explorer et appuyez sur le "+" pour **ajouter un connection SSH**. Dans la barre de recherche en haut tapez : `user@nom_du_robot.local`

Exemples :

- (a) `user@grkit-007.local` - pour le robot 7
- (b) `user@grkit-011.local` - pour le robot 11

Le mot de passe est **ROSKitGR** pour tous les robots. Pour finir, sélectionnez le **répertoire** dans lequel vous voulez travailler. Ici il s'agit de `/home/user/ros-kit_ws/src`. Une fois dedans vous verrez tous les packages ros2 déjà installés.

Initialisation Docker : Avant toute chose nous allons modifier le `.bashrc` afin qu'à chaque lancement notre conteneur soit bien sourcé. Rendez-vous dans le conteneur avec VSCode (en bas de la figure 7.c). faite dérouler, jusqu'à `/home/user/.bashrc`. en bas du fichier vous pouvez inscrire :

```
source /opt/ros/humble/setup.bash
source install/setup.sh
```

De cette façon à chaque fois que vous entrerez dans un conteneur il sera prêt à l'utilisation.

Ensuite accédez au conteneur :

```
docker exec -it gr-p247-humble-container bash
# Verifiez que les topics sont publiés :
ros2 topic list
```

2 Test de Navigation Autonome Simple

Cette section a pour objectif de vous faire comprendre les mécanismes fondamentaux de la navigation robotique **sans utiliser Nav2**. Vous allez programmer directement le comportement du robot en utilisant l'odométrie et le LiDAR.

2.1 Objectif

Le robot doit être capable de :

- Avancer en continu dans une pièce
- Détecter les obstacles grâce au LiDAR
- Choisir la meilleure direction pour les éviter
- S'arrêter proprement en cas de danger

2.2 Création du package ROS2

Avant de coder, il faut créer un package ROS2. Contrairement à un simple dossier Python, un package ROS2 a une structure obligatoire que la commande suivante génère automatiquement :

```
cd ~/roscit_ws/src
ros2 pkg create robot_control \
  --build-type ament_python \
  --dependencies rclpy geometry_msgs nav_msgs sensor_msgs
```

Cela crée la structure suivante :

```
robot_control/
robot_control/
  __init__.py
  mon_noeud.py      # votre code ici
  package.xml       # dependances ROS2
  setup.cfg
  setup.py          # declare les entry points
```

Pour que ROS2 reconnaisse votre nœud, déclarez-le dans `setup.py` :

```
entry_points={
  'console_scripts': [
    'mon_noeud = robot_control.mon_noeud:main',
  ],
},
```

Puis compilez et sourcez :


```
cd ~/roscit_ws
colcon build --packages-select robot_control --symlink-install
source install/setup.bash
ros2 run robot_control mon_noeud
```

2.3 Architecture du programme

Le programme repose sur trois mécanismes ROS2 fondamentaux :

2.3.1 Publisher

Un **publisher** envoie des messages sur un topic. Pour faire bouger le robot, on publie des messages `Twist` sur `/cmd_vel`.

Le message `Twist` contient 6 paramètres de vitesse :

- `linear.x` — avancer / reculer (m/s)
- `linear.y` — déplacement latéral
- `linear.z` — altitude (drones)
- `angular.x` — roll
- `angular.y` — pitch
- `angular.z` — yaw, rotation sur place (rad/s)

2.3.2 Subscribers et callbacks

Un **subscriber** écoute un topic et appelle une fonction (*callback*) à chaque nouveau message reçu. Le callback met à jour les données du nœud en temps réel.

Deux subscribers sont nécessaires :

- `/odom` — pour connaître la position et l'orientation du robot
- `/rslidar_points` — pour détecter les obstacles

Important : les callbacks ne s'exécutent que si `rclpy.spin_once()` ou `rclpy.spin()` est appelé. Sans cela, les données ne sont jamais mises à jour.

2.3.3 Timer

Un **timer** appelle une fonction à intervalle régulier. La boucle principale est appelée toutes les 0.1 secondes — c'est elle qui envoie les commandes au robot en continu (nécessaire pour éviter l'arrêt automatique de sécurité du Scout).

```
self.timer = self.create_timer(0.1, self.boucle)
```

2.4 Odométrie — position et orientation

Le topic `/odom` publie la position (`x`, `y`) et l'orientation du robot sous forme de quaternion (`x`, `y`, `z`, `w`).

Pour obtenir le **yaw** (rotation autour de l'axe vertical) à partir du quaternion :

```
siny = 2.0 * (w * z + x * y)
cosy = 1.0 - 2.0 * (y * y + z * z)
yaw = atan2(siny, cosy)
```

La **distance parcourue** depuis un point de départ (x_0, y_0) :

```
distance = sqrt((x - x0)**2 + (y - y0)**2)
```

L'**angle parcouru** depuis un yaw de départ θ_0 :

```
diff = yaw - yaw0
# Normaliser entre -pi et pi
while diff > math.pi: diff -= 2 * math.pi
while diff < -math.pi: diff += 2 * math.pi
```

2.5 Traitement du LiDAR

Le LiDAR publie un nuage de points 3D sur `/rslidar_points`. Chaque point contient les coordonnées (x, y, z) dans le repère du robot :

- x positif = devant, x négatif = derrière
- y positif = gauche, y négatif = droite
- z = hauteur

Pour lire les points :

```
import sensor_msgs_py.point_cloud2 as pc2

for point in pc2.read_points(msg,
    field_names=('x', 'y', 'z'), skip_nans=True):
    x, y, z = point
```

On filtre les points par hauteur pour ignorer le sol et le plafond, puis on calcule la distance minimale dans chaque secteur (devant, derrière, gauche, droite). Si cette distance est inférieure à un seuil de sécurité, un obstacle est signalé.

2.6 Logique de navigation

La boucle principale gère deux comportements :

1. **Mouvement** — si le robot doit avancer et qu'aucun obstacle n'est détecté devant, on publie une vitesse positive sur `linear.x`. On s'arrête quand la distance cible est atteinte (mesurée par l'odom).
2. **Rotation** — on publie une vitesse sur `angular.z` jusqu'à ce que l'angle cible soit atteint (mesuré par le yaw de l'odom).

Dans le `main`, la logique d'exploration est simple :

- Obstacle devant → calculer la meilleure direction (gauche ou droite, selon la distance maximale disponible) → tourner d'un angle
- Voie libre → avancer d'une certaine distance
- Répéter indéfiniment

2.7 Code source

Le code a trou de ce TP est disponible sur GitHub à l'adresse suivante :

https://github.com/MatthiasVermot-Desroches/Polytech-Scout-Agilex-GR-TP/blob/main/Chavaroche/Document%20Code/test_nav_a_trou.py

Essayez de l'implémenter vous-même en suivant les explications ci-dessus. Les éléments clés à implémenter sont :

1. Le callback `odom_callback` — mise à jour de `x`, `y`, `yaw`
2. Le callback `lidar_callback` — calcul des distances par secteur
3. La méthode `boucle` — logique de mouvement et d'arrêt
4. La méthode `meilleure_direction` — choix gauche ou droite
5. Le `main` — boucle d'exploration avec `spin_once`

2.8 Vérifications

Avant de lancer le programme, vérifiez que les topics nécessaires sont bien publiés :

```
ros2 topic hz /odom          # doit afficher ~50 Hz
ros2 topic hz /rslidar_points # doit afficher ~10-15 Hz
ros2 topic echo /cmd_vel     # verifier les commandes envoyees
```

Important : vérifiez que la manette est en mode autonome (interrupteur n°2 en position haute). Rappel, cette interrupteur change le mode de communication du scout mini. En position haute, le scout mini communique en BUS sur le CAN0 avec la partie haute du robot, l'ordinateur de bord. Quand l'interrupteur est abaissé d'un cran, le robot ne reçoit d'ordre que de la télécommande. Si le robot ne bouge pas malgré les commandes correctes sur `/cmd_vel`, vérifiez `control_mode` dans `/scout_status` — il doit valoir 1.

```
ros2 topic echo /scout_status --once | grep control_mode
```